100 DAYS OF MACHINE LEARNING

DAY 4

Tensors, Shape & Rank • Anaconda • Jupyter • Google Colab

Official playlist study workbook

Video 11: What are Tensors | Tensor In-depth Explanation | Tensor in Machine Learning
Video 12: Installing Anaconda For Data Science | Jupyter Notebook for Machine Learning | Google Colab for ML

Videos 11 and 12 are approximately 41 and 37 minutes respectively (about 78 minutes combined; exact duration can vary).

Beginner-friendly explanations, Roman Urdu reminders, comparison tables, worked examples, exercises, answer key, and a companion Python practice file.

How to use this workbook

• **Read:** understand the concept before memorizing the terminology.

• **Predict:** calculate shapes by hand before checking examples.

• **Practise:** run the companion Python file; it uses only the Python standard library.

• **Review:** complete the exercises and checklist before moving to Day 5.

Learning outcomes

• Explain tensors, scalar, vector, matrix, shape, rank, and axes.

• Read common shapes and explain what each dimension represents.

• Recognize irregular (ragged) nested data that does not form a rectangular matrix.

• Distinguish Python, Anaconda, Jupyter Notebook, and Google Colab.

• Run a small Python practice script.

Roman Urdu: Aaj ka maqsad data ki dimensions samajhna aur Python code likhne/chalane ke tools ka farq seekhna hai.

Part 1 — What is a tensor?

A **tensor** is a general structure used to represent numerical data across one or more dimensions. In machine learning, tensor-like arrays can hold inputs, labels, parameters, and intermediate results. Scalars, vectors, and matrices are simple cases of data organized across different numbers of dimensions.

Precision: a nested Python list can demonstrate a tensor's structure, but it is not the same optimized numerical object as a NumPy array or a PyTorch/TensorFlow tensor. Those libraries provide defined dtypes and efficient numerical operations.

Structure	Common rank	Example	Meaning
Scalar	0	7	One value; shape commonly ()
Vector	1	[10, 20, 30]	One axis with three values; shape (3,)
Matrix	2	[[1,2,3],[4,5,6]]	Rows and columns; shape (2, 3)
Higher-dimensional	3 or more	Stack of matrices or image batch	Several axes, such as samples, height, width, channels

Examples in plain language

Scalar: one student's score, 87.5. **Vector:** one student's marks in three subjects, [80, 91, 76]. **Matrix:** two students' marks in three subjects, [[80, 91, 76], [72, 85, 90]], with shape (2, 3).

Roman Urdu: Scalar aik value; vector aik line/list of values; matrix rows aur columns; higher-dimensional tensor mein mazeed axes hotay hain.

Part 2 — Shape, rank, axes, and number of elements

Shape gives the size along each axis. For a regular matrix, shape (2, 3) means two rows and three columns. **Rank** commonly means the number of dimensions/axes in numerical array libraries. Rank is not a measure of a model's quality or accuracy.

Example	Shape	Rank	Interpretation
Scalar 7	()	0	One value
[4, 8, 9]	(3,)	1	Three entries on one axis
2 rows × 3 columns	(2, 3)	2	Six entries

Example	Shape	Rank	Interpretation
100 students × 5 subjects	(100, 5)	2	100 student rows and 5 subject columns
32 RGB images, 64×64	(32, 64, 64, 3)	4	Batch, height, width, channels
12 grayscale images, 28×28	(12, 28, 28)	3	Batch, height, width

Worked example: 100 students, 5 subjects

If every row represents a student and every column a subject, the matrix shape is **(100, 5)**. The first dimension has 100 rows; the second has 5 columns. If every position has a mark, there are 100 × 5 = **500 entries**.

Roman Urdu: Pehla number students/rows (100), doosra subjects/columns (5). Is liye shape (100, 5), aur total marks entries 500 hain.

Image example: channels-last ordering

A batch of 32 colour images, each 64 pixels high and wide, with 3 colour channels, may use shape **(32, 64, 64, 3)**. Some frameworks/configurations use channels-first, (32, 3, 64, 64). Always check the model's expected order.

Shape versus rank versus element count

Term	Definition	Example: (8, 4)
Shape	Size along each axis	(8, 4)
Rank / ndim	Number of axes	2
Number of elements	Total values in a dense array	8 × 4 = 32
Dtype	Kind of values stored	integer, float, Boolean

Part 3 — Dtypes and practical tensor uses

A data type (often called **dtype**) describes the kind of values stored. Common examples are integers, floating-point numbers, and Boolean values. The exact types and memory behaviour depend on the numerical library and device.

Type	Example	Possible use
Integer	0, 1, 100	Counts, some labels or class IDs
Floating point	0.25, 3.14	Measurements, probabilities, model calculations
Boolean	True / False	Masks and logical flags

Where tensor-shaped data appears

• **Tabular data:** rows are records; columns are features.

• **Images:** height, width, colour channels, and sometimes batch size.

• **Text:** token IDs or embeddings arranged by batch and sequence.

• **Audio/time series:** time steps, channels, or measured features.

• **Deep learning:** tensors store inputs, parameters, activations, gradients, and outputs.

CPU and GPU

Libraries may run numerical operations on a CPU or a supported GPU. GPUs can accelerate suitable parallel workloads, but not every task becomes faster and transferring data has a cost. For now, focus on understanding shape and meaning.

Python lists versus numerical tensors

Python lists are general-purpose containers. NumPy arrays and PyTorch/TensorFlow tensors are specialized numerical objects with defined data types and efficient operations. This workbook's code uses nested lists to teach shape without requiring extra packages.

Part 4 — The data-science toolchain

Name	What it is	Main use
Python	Programming language	Write instructions, functions, data-processing and ML code.
Anaconda	Python distribution plus environment/package tools	Set up a data-science environment and manage packages.
Jupyter Notebook	Interactive notebook interface/application	Run code in cells and combine outputs with written explanations.
Google Colab	Hosted notebook service in a browser	Try notebook code online with minimal local setup; service limits and connectivity apply.

Easy analogy

Python is the language. Jupyter is an interactive notebook. Colab is a hosted notebook service you can open in a browser. Anaconda helps provide and manage Python environments and packages.

Roman Urdu: Python zubaan hai; Jupyter notebook interface hai; Colab browser-based hosted notebook hai; Anaconda environments aur packages manage karne mein madad karta hai.

Part 5 — How to work in a notebook

A notebook contains cells. A **code cell** runs Python instructions; a **Markdown/text cell** explains the work. You can run one cell at a time and inspect its output immediately below it.

Example code cell

```
print('Hello, Machine Learning!')
students = 100
subjects = 5
shape = (students, subjects)
print('Marks matrix shape:', shape)
```

Expected output

```
Hello, Machine Learning!
Marks matrix shape: (100, 5)
```

A beginner workflow

• Open a notebook in Jupyter or Google Colab.

• Create a code cell and type a small Python statement.

• Run the cell, read the output, then change one thing and run again.

• Save the notebook and understand where any external files are stored.

• Keep credentials and private student data out of public/shared notebooks.

Which tool may fit?

Need	Possible choice
Local environment and package management	Anaconda or another environment/package manager
Interactive notebook on your computer	Jupyter Notebook, installed in a suitable environment
Try notebook code in a browser with little setup	Google Colab, subject to connectivity and service limits
Repeatable project work	Document the environment and dependencies; save files carefully

Common cautions

• Installing packages globally can cause version conflicts; separate environments can help.

• A notebook may rely on local files that are not automatically available on another machine.

• Hosted sessions may reset or disconnect; save important work.

• Never put passwords, private keys, or sensitive records in a shared notebook.

Part 6 — Companion Python practice file

CampusX_ML_Day_04_Practice.py is a self-contained practice script. It uses only the standard library, demonstrates shapes and ranks, checks rectangular data, calculates the (100, 5) marks shape, and summarizes the notebook tools.

Run from a terminal in the file's folder:

```
python CampusX_ML_Day_04_Practice.py
```

If your system uses python3 instead, run: `python3 CampusX_ML_Day_04_Practice.py`. If the command is not recognized, Python may not be installed or may not be on PATH. You can also copy examples into notebook cells.

Part 7 — Practice questions

Try to answer these before reading the key.

1. A 1D array contains 9 values. What are its shape and rank?
2. A matrix has 8 rows and 4 columns. Give its shape, rank, and element count.
3. What is the shape of marks for 100 students in 5 subjects, with one row per student?
4. Twelve grayscale images are 28×28 pixels. What is the batch-first shape and rank?
5. Sixteen RGB images are 32×32 pixels. Give a common channels-last shape.
6. Why is [[1, 2], [3]] not a regular rectangular matrix?
7. Explain shape, rank, and element count in one sentence each.
8. Which is the programming language: Python, Jupyter, or Colab?
9. Which tool may let you try notebook code in a browser with little local setup?
10. Give one benefit of separate Python environments.
11. True or false: tensor rank tells you whether predictions are accurate.
12. What must you check if a model expects channels-first but your data is channels-last?

Answer key with explanations

1. Shape (9,), rank 1: one axis with nine entries.
2. Shape (8, 4), rank 2, and 8×4 = 32 elements.
3. Shape (100, 5): 100 student rows and 5 subject columns.
4. Shape (12, 28, 28), rank 3: batch, height, width.
5. Shape (16, 32, 32, 3): batch, height, width, channels.
6. Row lengths differ: the first has two entries and the second has one.
7. Shape gives sizes along axes; rank counts axes; element count is the total values (for (8, 4), 32).
8. Python is the language; Jupyter and Colab provide notebook experiences.
9. Google Colab may be convenient, depending on connectivity and service limitations.
10. Separate environments can reduce dependency/version conflicts between projects.
11. False. Rank is structural, not an accuracy score.
12. Check the required axis ordering and reorder/convert data if necessary.

Part 8 — Revision sheet

Term	Remember
Tensor	A numerical data structure that can have multiple dimensions.
Scalar / vector / matrix	Commonly 0D / 1D / 2D structures.
Shape	Size along each axis, e.g. (100, 5).
Rank	Number of dimensions/axes in common array-library terminology.
Elements	Total values; (100, 5) contains 500 positions if fully populated.
Dtype	The kind of stored values, such as integer, float, or Boolean.
Python	Programming language.
Anaconda	Distribution and tools for environments/packages.
Jupyter	Interactive notebook interface/application.
Google Colab	Hosted notebook service in a browser.

Checklist — tick each item when you can explain it without reading

• I can define a tensor in simple words.

• I can identify scalar, vector, matrix, and higher-dimensional examples.

• I can explain shape (100, 5) and why it represents 100 students and 5 subjects.

• I know shape (2, 3) has rank 2 and 6 elements.

• I understand why ragged data can cause shape problems.

• I can distinguish Python, Anaconda, Jupyter, and Colab.

• I can run the companion practice file.

Memory trick

Shape = sizes; rank = number of axes; elements = how many values. For (100, 5), there are two axes, sized 100 and 5, and 500 positions if the matrix is fully populated.

Roman Urdu recap

Tensor numerical data ko organize karne ka aik general tareeqa hai. Shape har dimension ka size batata hai; rank dimensions ki tadaad hai. 100 students aur 5 subjects ki matrix ka shape (100, 5) hoga. Python language hai, Jupyter notebook interface hai, Colab browser-based notebook service hai, aur Anaconda environment/packages manage karne mein madad karta hai.

You marked videos 11 and 12 complete. Review the shape-versus-rank distinction and the marks-matrix example before moving to Day 5.

Educational examples are simplified. Library APIs and runtime behaviour can vary by version.